

Metode Numerik: Penyelesaian SPL secara Iteratif (Gauss-Seidel)

Catatan Kuliah Komputasi Numerik

16 Juni 2026

1 Pendahuluan

Berbeda dengan metode eliminasi langsung, metode ****Iterasi Gauss-Seidel**** menyelesaikan Sistem Persamaan Linear (SPL) secara aproksimasi berulang. Metode ini sangat efisien untuk SPL berukuran besar dengan matriks koefisien yang dominan secara diagonal (*diagonally dominant*).

Kita memulai proses dengan menentukan tebakan awal untuk semua variabel, yang umumnya diinisialisasi dengan nilai nol:

$$x_1^{(0)} = 0, \quad x_2^{(0)} = 0, \quad \dots, \quad x_n^{(0)} = 0 \quad (1)$$

2 Algoritma dan Formulasi Matematika

Untuk SPL dengan n persamaan, kita mengisolasi setiap variabel x_i pada persamaan ke- i . Rumus umum untuk iterasi ke- $(k+1)$ adalah:

$$x_i^{(k+1)} = \frac{b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)}}{a_{ii}} \quad (2)$$

2.1 Karakteristik Substitusi Balik Berulang

Kunci utama Gauss-Seidel yang membedakannya dengan metode Jacobi adalah ****penggunaan langsung nilai terbaru****. Begitu nilai $x_1^{(k+1)}$ didapatkan dari baris pertama, nilai baru tersebut langsung dipakai untuk menghitung $x_2^{(k+1)}$ di baris kedua pada iterasi yang sama. Proses ini terus berulang secara iteratif sampai selisih nilai antar-iterasi lebih kecil dari toleransi error yang ditentukan (ϵ).

3 Analisis Kasus Khusus ($m < n$)

Jika jumlah persamaan lebih sedikit daripada jumlah variabel ($m < n$), metode iteratif seperti Gauss-Seidel akan menghadapi masalah serius:

- **Matrix Underdetermined:** Kita tidak memiliki cukup persamaan untuk mengisolasi setiap variabel di sisi kiri rumus iterasi.
- **Divergensi:** Karena adanya variabel bebas yang tidak terikat oleh baris pivot, nilai iterasi kemungkinan besar akan terus membesar tanpa batas (*diverge*) atau terjebak dalam lingkaran iterasi tak hingga tanpa pernah konvergen ke solusi manapun.

4 Implementasi Code (Python)

Berikut adalah implementasi metode Gauss-Seidel dengan tebakan awal nol dan ukuran tabulasi 2 spasi:

```

1 import numpy as np
2
3 def gauss_seidel(A, B, x_awal, tol=1e-5, max_iter=100):
4     A = A.astype(float)
5     B = B.astype(float)
6     x = x_awal.copy().astype(float)
7     n = len(B)
8
9     for k in range(max_iter):
10         x_lama = x.copy()
11
12         for i in range(n):
13             # Hitung sigma untuk variabel yang sudah diupdate dan belum
14             sigma = 0.0
15             for j in range(n):
16                 if j != i:
17                     sigma += A[i][j] * x[j]
18
19             # Ambil nilai terbaru langsung (Back Substitution internal)
20             x[i] = (B[i] - sigma) / A[i][i]
21
22         # Cek konvergensi menggunakan norm
23         if np.linalg.norm(x - x_lama, ord=np.inf) < tol:
24             print(f"Konvergen pada iterasi ke-{k+1}")
25             return x
26
27     print("Metode tidak konvergen dalam batas maksimum iterasi")
28     return x
29
30 # Contoh Kasus
31 A = np.array([[4, 1, -1], [2, 7, 1], [1, -3, 12]])
32 B = np.array([3, 19, 31])
33 # Tebakan awal semua variabel = 0
34 tebakan_awal = np.array([0, 0, 0])
35
36 solusi = gauss_seidel(A, B, tebakan_awal)
37 print("Solusi:", solusi)

```

Listing 1: Implementasi Iterasi Gauss-Seidel